

Yes. Since this is an **AIOps practical assessment**, the safest approach is to use Copilot as a **coding/debugging assistant**, but you should personally understand and verify every change. Don't ask Copilot to blindly "complete the whole assessment."

I'd follow this exact workflow in Codespaces.

1. First open the repository and DON'T change anything

Open the GitHub repository in Codespaces.

Then in Copilot Chat, give it this prompt:

```
I am working on an AIOps practical assessment.
```

```
Do NOT modify any files yet.
```

```
First inspect the entire repository and explain:
```

1. Project structure
2. Purpose of every important file/folder
3. Where operational data is generated/stored
4. Where metrics and logs are handled
5. Where anomaly detection happens
6. Where events are generated
7. Where the producer is implemented
8. What represents the event topic/queue
9. Where the consumer is implemented
10. Where the final AIOps processing/output happens
11. How data flows end-to-end through the project
12. How I should run the complete workflow
13. What tests/validation commands already exist

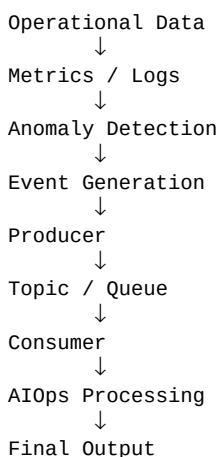
```
Also identify the main entry-point files and configuration files.
```

```
Do not make any changes. I want to understand the existing architecture first.
```

Important: Read Copilot's answer yourself. Compare it with the actual files.

2. Make an architecture map

You want to understand this pipeline:



Ask Copilot:

Based only on the existing repository, trace one piece of operational data through the entire system.

Show me the exact:

file → function/class → next file → function/class

for:

Operational Data

→ Metrics/Logs

→ Anomaly Detection

→ Event Generation

→ Producer

→ Topic

→ Consumer

→ Final AI0ps Output

Do not modify anything.

This gives you a very useful map for the assessment.

3. Run the project BEFORE fixing anything

This is important.

Ask Copilot:

Based on the repository, tell me the exact commands I should run to execute the existing workflow.

Do not modify files.

Include:

- dependency installation
- environment setup if required
- tests
- individual components if they can be run separately
- complete end-to-end workflow

Explain what output I should expect from each command.

Then run the commands **yourself in the Codespace terminal**.

For example, you may see something like:

```
pip install -r requirements.txt
```

and then:

```
python something.py
```

But **don't assume these filenames**. Let the repository tell you.

4. Understand the operational data

Now specifically investigate the data.

Copilot prompt:

Inspect the operational data used by this project.

Tell me:

1. What metrics are present?
2. What log information is present?
3. What represents normal behaviour?
4. What represents unusual/anomalous behaviour?
5. Which fields/features are used by the anomaly detector?
6. What thresholds or model logic determine an anomaly?

Show me a few concrete examples from the repository/data and explain why they are normal or anomalous.

Do not change anything.

You should understand things like:

CPU = 25%
Memory = 40%
Latency = 100ms

might be normal, whereas:

CPU = 98%
Memory = 95%
Latency = 2500ms

might trigger an anomaly.

Don't invent these values—use whatever your repository actually contains.

5. Run anomaly detection separately

This is one of the most important parts of your assessment.

Ask Copilot:

Find the exact code responsible for anomaly detection.

Tell me:

- file name
- function/class
- input
- output
- detection logic/model
- how an anomaly is represented

Then tell me the exact terminal command to run **ONLY** the anomaly detection portion.

Do not modify anything.

Run it.

You want to actually see something like:

Metric: CPU
Value: ...
Status: NORMAL

and potentially:

```
Metric: CPU
Value: ...
Status: ANOMALY
```

Record the output.

6. Run event generation

Next:

Find the code that converts an anomaly into an event.

Explain:

1. What triggers event generation?
2. What fields are included in the event?
3. What does the generated event look like?
4. How can I run this component independently?

Do not modify anything.

Then run it.

You might get something conceptually like:

```
{
  "service": "...",
  "metric": "...",
  "value": "...",
  "severity": "...",
  "timestamp": "..."
}
```

Again, use **your actual repository output**.

7. Understand Producer → Topic → Consumer

Now ask Copilot:

Trace the event after it is generated.

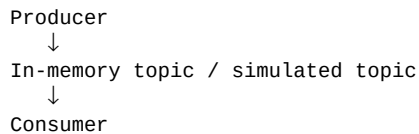
Identify:

- producer file/function
- topic/queue implementation
- consumer file/function
- how the event is passed from producer to consumer
- whether this is a real Kafka implementation or a simulation/mock
- exact commands to run producer and consumer

Do not modify anything.

Your assessment explicitly says you don't need real Kafka.

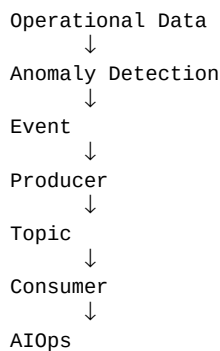
So there may be something like:



Understand exactly what the repository does.

8. Run the complete workflow

Once you understand each part:



ask:

Now give me the exact commands to execute the complete end-to-end workflow using the existing architecture.

Do not change anything.

Also tell me what evidence/output I should verify at each stage to confirm that:

1. operational data was processed
2. anomaly was detected
3. event was created
4. producer published it
5. topic received it
6. consumer consumed it
7. final AIops processing occurred

Run it.

Take screenshots of important outputs.

9. Now investigate the debugging issue

Your assessment says there will be troubleshooting.

Don't immediately ask Copilot to fix it.

First reproduce it.

If something fails, ask:

The workflow is failing.

Do NOT modify anything yet.

Analyse the error and determine:

1. Exact failing component
2. Exact file
3. Exact function/class
4. Root cause
5. Why the current implementation fails
6. What the smallest possible correction would be
7. How we can verify the correction

Do not make changes yet.

This is much better than:

> "Fix everything."

Because the assessment wants you to identify the **affected component and cause**.

10. Then ask Copilot for the smallest fix

After you understand the problem:

Apply only the minimal correction required to fix the identified issue.

Important:

- Preserve the existing architecture.
- Do not rewrite working components.
- Do not add unnecessary dependencies.
- Do not change unrelated files.
- Keep the existing coding style.
- Do not remove existing functionality.

After making the change, explain exactly what was changed and why.

Then inspect the diff:

```
git diff
```

This is extremely important.

11. Run tests immediately

After the fix:

```
git status
```

Then run the project's tests.

Ask Copilot:

Identify all existing validation/test commands in this repository.

I have just made a minimal fix. Tell me which tests I should run to validate:

1. anomaly detection
2. event generation
3. producer
4. consumer
5. final AIOps processing
6. complete workflow

Run them.

If a test fails, **don't let Copilot randomly patch it**. Investigate the failure first.

12. Run the complete workflow AGAIN

This is required.

You want evidence that:

Before fix:

- workflow fails

After fix:

- anomaly detected
- event generated
- producer works
- topic works
- consumer works
- AIOps output generated

Your README can then describe this clearly.

13. Git commits — don't make one giant commit

I recommend small logical commits.

For example:

Commit 1 — Initial understanding

```
git status
git add .
git commit -m "Document initial AIOps workflow analysis"
git push
```

But **only if you actually made a documentation change**.

Don't create meaningless commits just for the sake of commits.

Commit 2 — Fix

After the actual bug fix:

```
git status
git diff
git add <changed-files>
git commit -m "Fix event processing workflow"
git push
```

Commit 3 — README/evidence

```
git add README.md
git commit -m "Document AIOps workflow and validation"
git push
```

Use commit messages that describe what actually changed.

14. Before every commit

Use this sequence:

```
git status
git diff
```

Then:

```
git add <specific files>
git commit -m "... "
git push
```

I recommend **not** doing:

```
git add .
```

unless you've checked the repository and are sure you aren't accidentally committing things like:

```
.env
credentials
large files
temporary files
logs
venv/
__pycache__/
```

Check:

```
git status
```

first.

15. README — make it human, not AI-looking

Don't make the README sound like:

> "This project demonstrates a sophisticated end-to-end AIOps architecture leveraging cutting-edge..."

That's exactly the kind of thing that looks AI-generated.

A good structure:

```
# AIOps Practical Assessment
```

```
## 1. Overview
```

Short explanation of what the repository does.

```
## 2. Existing Workflow
```

Operational Data

↓

Anomaly Detection

↓

Event Generation

↓

Producer

↓

Topic

↓

Consumer

↓

AIOps Processing

```
## 3. Repository Components
```

Component	File	Purpose
Operational Data	...	...
Anomaly Detection	...	...
Event Generation	...	...
Producer	...	...
Topic	...	...
Consumer	...	...
AIOps Processing	...	...

```
## 4. Observations
```

Explain what you observed in the operational data.

Mention:

- normal behaviour
- unusual behaviour
- relevant metrics
- logs
- anomaly conditions

```
## 5. Issue Found
```

Explain:

- what failed
- where it failed
- error observed
- root cause

```
## 6. Correction
```

Explain exactly what you changed and why.

```
## 7. Validation
```

Commands used:

...

Then explain the result.

8. End-to-End Result

Explain how the event moved through the system.

9. Reproduction Steps

1. Clone repository
2. Install dependencies
3. Run ...
4. Run ...
5. Verify ...

10. Evidence

Screenshots/output showing:

- anomaly detection
- event generation
- producer
- consumer
- final output
- successful tests

16. Ask Copilot to help write the README, but give it your observations

After you've actually done the work, use this prompt:

I have completed the AIOps assessment and verified the workflow.

Using ONLY the actual repository structure, commands I ran, observed outputs, and changes I made, help me draft the README.

Requirements:

- Write like a student documenting practical work.
- Keep the language simple and natural.
- Do not exaggerate the project.
- Do not invent results, metrics, files, commands, or observations.
- Clearly distinguish observations from explanations.
- Mention the actual issue I found and its root cause.
- Explain the correction I made.
- Include the actual validation commands.
- Include reproduction steps.
- Include the end-to-end workflow.
- Keep it concise and technically accurate.
- Do not use generic AI marketing language.

Then **read every line yourself**.

If Copilot writes something you didn't actually observe, delete it.

17. Make your README sound human

Instead of:

> "The system successfully demonstrates a robust and scalable AIOps pipeline."

Write:

> "I first ran the existing workflow to understand how an operational metric moves through the system. The anomaly detector identifies unusual values and generates an event, which is then passed to the producer and consumer before the final AIOps processing."

Instead of:

> "Comprehensive validation was performed."

Write:

> "After applying the fix, I reran the workflow and the event reached the consumer successfully. I also ran the available tests to verify that the change did not break the existing flow."

That's much more natural.

18. Screenshots you should keep

I'd keep evidence for at least:

Screenshot 1

Repository structure / important files.

Screenshot 2

Operational data + anomaly detection output.

Screenshot 3

Generated event.

Screenshot 4

Producer/topic/consumer output.

Screenshot 5

Final AIOps output.

Screenshot 6

The error before fixing.

Screenshot 7

`git diff` showing your correction.

Screenshot 8

Tests passing.

Screenshot 9

Final end-to-end successful run.

Don't put 20 screenshots in README. Pick the useful ones.

19. One very useful Copilot rule

Throughout the assessment, use this pattern:

First ask:

> **"Explain what's happening. Don't change anything."**

Then:

> **"Identify the root cause."**

Then:

> **"Suggest the minimal fix."**

Then:

> **"Apply only that fix."**

Then:

> **"Show me the diff."**

Then:

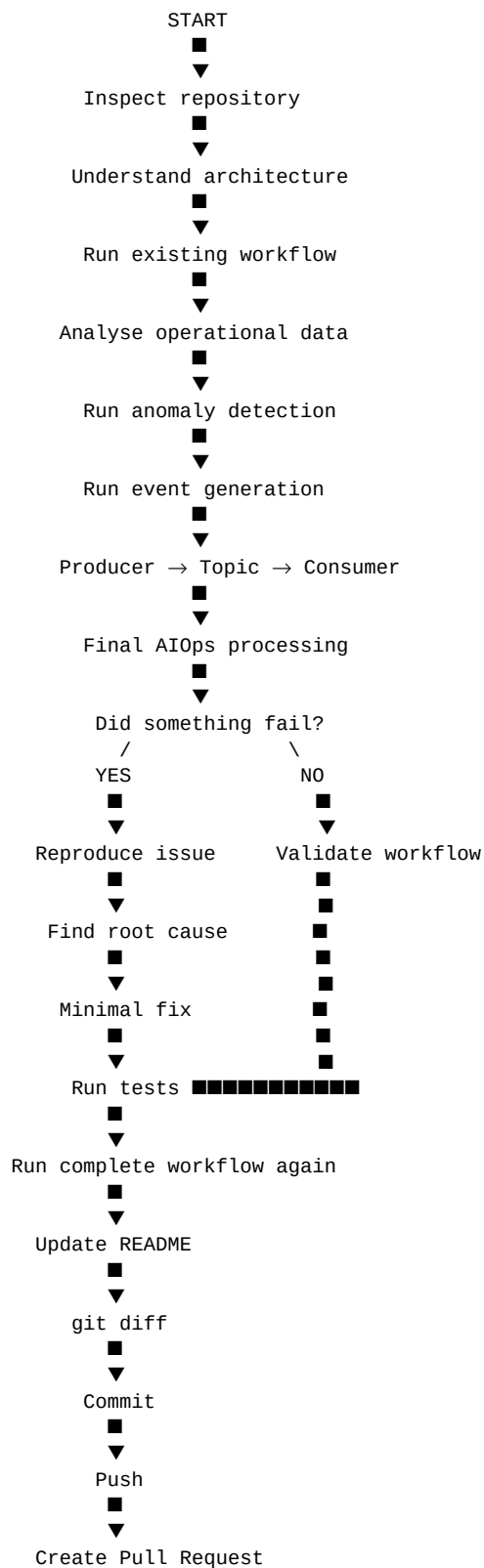
> **"Tell me how to validate it."**

Then **you run the validation.**

This keeps you in control and makes it much easier to explain your work if the evaluator asks questions.

Your complete assessment flow

Think of your Codespace session like this:



One important thing

Since this is an **assessment**, don't paste the entire repository into Copilot and tell it **"do the MSE for me."** Use Copilot for navigation, explanation, debugging, commands, and drafting documentation, but **you should run the commands, inspect the outputs, understand the root cause, and verify the final result yourself.**

If you give me the **GitHub repository URL**, I can also walk you through it **file-by-file**: first identify the architecture, then tell you exactly what to run for anomaly detection/event generation/producer/consumer, what output to look for, and

where the likely debugging task is—without blindly modifying the project.